

Universidade do Minho

**Licenciatura em Engenharia
Informática**

Redes de Computadores

Relatório do Trabalho Prático Nº2

José Ornelas a111790

Tiago Carvalhido a110491

Tiago Pereira a112042

6 de maio de 2026

1. Introdução	3
2. Arquitetura e Implementação	3
2.1. O Motor de Captura	3
2.2. A Lógica de Filtragem	4
2.3. A Lógica de Desencapsulamento	5
3. Funcionalidades do Sniffer	6
3.1. Configuração Interativa e Captura em Tempo Real	6
3.2. Modos de Execução	7
3.3. Filtros e Parâmetros	9
3.4. Logging de Dados	10
3.5. Funcionalidades adicionais	11
3.6. Rastreamento de Estado (State Tracking) e Memória de Correlação	12
4. Protocolos Analisados	12
4.1. Camada de Ligação de Dados e ARP	12
4.2. Camada de Rede e Inspeção de Encapsulamento (IPv4/IPv6)	13
4.3. ICMP	13
4.4. Transporte (TCP) e Inspeção Aplicacional (HTTP/HTTPS)	14
4.5. UDP e Dissecção de DNS	14
5. Demonstração e Teste no CORE (Parte A)	15
6. Demonstração na Interface Real (Parte B)	18
6.1. Execução do Sniffer na Interface Real	18
6.2. Captura e Observação de Tráfego Real	21
6.2.1 Captura de tráfego ICMP	22
6.2.2 Captura de tráfego DNS	23
6.2.3 Captura de tráfego HTTPS	24
6.3. Demonstração dos Filtros	26
6.4. Obstáculos Encontrados	28
6.5. Resumo Estatístico	29
7. Exportação e Tratamento de Dados	30

1. Introdução

No âmbito da unidade curricular de Redes de Computadores, este relatório reflete o desenvolvimento de um Packet Sniffer e respetivo analisador de tráfego. A criação do sniffer foi desenvolvida em linguagem Python, recorrendo à biblioteca *scapy*, com o objetivo de capturar, filtrar e registar pacotes de rede em tempo real.

Para viabilizar uma análise abrangente, a ferramenta opera num nível privilegiado do sistema operativo. Ao requerer permissões do administrador, o sniffer configura a interface de rede para o modo promíscuo. Assim, não são descartados pacotes e a nossa aplicação interceta a totalidade de comunicações que circulam no meio físico.

A fim de comprovar o seu funcionamento, testámos o nosso sniffer em dois cenários diferentes, num emulador CORE e numa interface física real. Em relação ao CORE, criámos um ambiente de rede controlado para observar o tráfego de forma isolada. Na interface real, lidámos com um elevado volume de tráfego do dia a dia de uma rede de Internet, demonstrando a capacidade do sniffer nessas condições e na aplicação dos filtros.

2. Arquitetura e Implementação

Para que o sniffer funcione de forma eficiente, a sua arquitetura foi dividida em três pilares principais: o motor de captura, o sistema de filtragem e a lógica de desencapsulamento dos pacotes.

2.1. O Motor de Captura

O motor da nossa aplicação é alimentado pela função *sniff()*, da biblioteca *scapy*. Para uma captura e análise em tempo real, utilizamos um mecanismo de *callback* (através do parâmetro *prn*). Este mecanismo faz com que, mal o pacote é intercetado, a biblioteca invoque imediatamente a função *processar_pacote*. Esta função atua como porta de entrada, submetendo o pacote à função *aplicar_filtros* antes de permitir o seu avanço para a fase de identificação e output.

Esta abordagem permitiu-nos uma melhor gestão de memória pois não existe necessidade de manter o histórico de todo o tráfego capturado. Por esse motivo, configuramos a função *sniff()* com o argumento *store = False*.

Esta configuração garante que cada pacote é destruído da memória RAM após a sua análise e saída no terminal ou escrita no ficheiro de log. Sem esta decisão tomada, em cenários de tráfego intenso ou capturas de longa duração, a aplicação iria armazenar os pacotes progressivamente, causando esgotamento de memória e, por sua vez, colapso do sistema. Assim, garantimos um motor leve, fiável e capaz de processar pacotes durante o tempo que o utilizador desejar sem causar perdas de desempenho.

2.2. A Lógica de Filtragem

Para lidar com a enorme quantidade de pacotes interceptados pelo sniffer, implementamos um sistema de filtragem. Na nossa arquitetura de filtragem, optamos por utilizar dois processos distintos, o filtro BPF e o filtro de aplicação.

Em relação ao filtro BPF, o utilizador pode invocá-lo através do comando `-f` (por exemplo, `-f tcp port 80`, que captura apenas tráfego Web HTTP). A grande vantagem desta abordagem é a sua localização, pois atua no kernel do sistema operativo. O BPF funciona como uma barreira de entrada dos pacotes. Assim, se um pacote não corresponder à regra definida, é automaticamente descartado na placa de rede, não atingindo sequer o código principal da nossa aplicação.

Em relação ao filtro de aplicação, pode ser invocado por diferentes comandos (por exemplo, `--ip 10.0.0.1`, para isolar as comunicações de um dispositivo específico). Ao contrário do filtro BPF, que apenas se foca em cabeçalhos básicos (MAC, IP, portas), este segundo nível de filtragem permite inspecionar o interior do pacote, sendo necessariamente recebido pelo nosso programa e desencapsulado pelo *scapy*.

Apesar do filtro de aplicação ser mais lento e computacionalmente mais exigente, é estritamente necessário pois é ele que permite que o sniffer analise o payload e distinga os diferentes protocolos da camada de aplicação, como uma mensagem específica de DHCP ou um pedido de DNS. A combinação destas duas estratégias de filtragem garante que o nosso sistema rejeita o tráfego irrelevante com excelente desempenho, sem comprometer a sua capacidade de realizar inspeções profundas.

2.3. A Lógica de Desencapsulamento

Quando um pacote ultrapassa a filtragem com sucesso, entra na fase do desencapsulamento. Neste passo, entra a função *processar_pacote*, que adota uma abordagem *bottom-up* (baixo para cima), seguindo o modelo OSI.

Para implementar esta lógica, utilizamos a função *haslayer()* da biblioteca *scapy*. Assim, o código inspeciona a existência de cada cabeçalho sequencialmente, seguindo a ordem do modelo OSI.

O processo inicia-se na camada 2 (Ligação de Dados), onde a ligação verifica a existência do cabeçalho Ethernet (Ether). Validada esta camada, o sniffer extrai os endereços físicos (MAC) de origem e destino, que são essenciais para compreender a comunicação no meio local. Subindo o nível de abstração para a camada 3 (Rede), o código procura pelo cabeçalho IP. Confirmada a sua existência, procede-se à extração dos endereços lógicos, o que permite identificar os verdadeiros emissores e recetores na rede.

Em seguida, o processamento avança para a camada 4 (Transporte). Nesta fase, a função determina o protocolo utilizado pelo pacote (TCP ou UDP) para proceder à extração das respetivas portas de comunicação. A abordagem progressiva na análise da informação permite que o código só tente ler a Camada de Aplicação (Camada 7) após a confirmação bem-sucedida das camadas inferiores. Este rigoroso método sequencial provou ser crucial para prevenir falhas de execução, como a tentativa de extrair uma porta TCP de um pacote ICMP, o que resultaria no bloqueio da aplicação.

Consequentemente, asseguramos um processo de desencapsulamento robusto e à prova de erros, resultando num registo estruturado e perfeitamente legível para o utilizador.

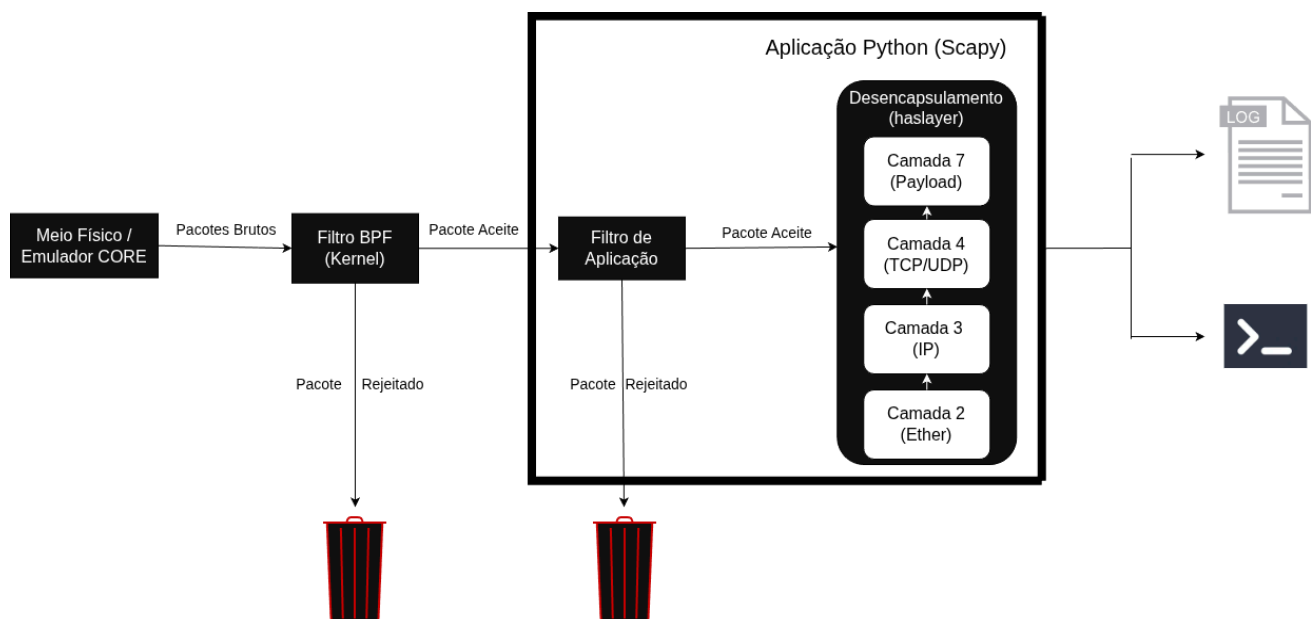


Figura 1 - Arquitetura do motor de captura.

3. Funcionalidades do Sniffer

O desenvolvimento desta ferramenta de análise de rede teve como principal objetivo a conceção de um utilitário capaz de **monitorar**, **capturar** e **interpretar** o tráfego de dados num dado domínio de colisão ou difusão. A aplicação foi integralmente desenvolvida em Python, tirando partido da flexibilidade e do poder da biblioteca **scapy**, uma das ferramentas mais conceituadas no meio académico e profissional para a manipulação de pacotes a baixo nível. Através de uma arquitetura procedimental, o nosso *sniffer* não se limita à escuta cega do meio físico, oferecendo um conjunto de funcionalidades que facilitam a filtragem, a legibilidade e a análise.

3.1. Configuração Interativa e Captura em Tempo Real

A mecânica central de intercessão do tráfego assenta na função **sniff()**. Para dotar o programa de uma forte componente interativa, implementámos um módulo de arranque inteligente. Caso não sejam passados argumentos nativos, a função **perguntar_configuracao()** invoca o método **get_if_list()** para varrer o sistema operativo e apresentar de forma clara todas as interfaces de rede disponíveis, permitindo a seleção através de um identificador numérico. No terminal, os pacotes destacam-se através da implementação de códigos de cor ANSI, que mapeiam tons distintos para diferentes protocolos, reduzindo drasticamente a carga cognitiva durante uma análise em tempo real.

```
===== CONFIGURAÇÃO DO SNIFFER =====

Interfaces disponíveis:
0 - lo
1 - eth0

Escolhe a interface pelo número: 1

Quantos pacotes queres capturar? [0 para infinito]:

Protocolo a filtrar (ex: ARP, ICMP, TCP, DNS) [Enter para todos]:

Filtro BPF (ex: tcp, udp, port 53) [Enter para nenhum]:

Filtrar por IP [Enter para nenhum]:

Filtrar por MAC [Enter para nenhum]:

Guardar log? [s/n]:

===== CONFIGURAÇÃO FINAL =====
Interface: eth0
Pacotes: infinito
Protocolo: Todos
Filtro BPF: Nenhum
IP: Nenhum
MAC: Nenhum
Log: Desativado
=====
```

Figura 2 - Menu inicial onde se pergunta todos os filtros possíveis do sniffer

3.2. Modos de Execução

O ciclo de vida da captura foi desenhado para suportar dois modos de execução distintos:

- **Captura Contínua:** Por omissão, o **sniffer** entra num ciclo infinito. Toda a rotina está envolvida num bloco de tratamento de exceções. Se o utilizador interromper a execução (via **Ctrl+C**), o programa termina de forma segura, transitando o fluxo para o encerramento de ficheiros e apresentação do sumário.
- **Modo Limitado (Quota):** Através do argumento **--count** (ou da configuração no menu), o utilizador define um teto máximo numérico de pacotes, encerrando a execução autonomamente ao atingir esse valor.

```
jordan@PCTiago:~/UMinho/RC/REDES_SNIFFER$ sudo python3 sniffer.py --count 2
===== Packet Sniffer RC-TP2 =====
Interface: eth0
Filtro BPF: Nenhum
Filtro protocolo: Nenhum
Filtro IP: Nenhum
Filtro MAC: Nenhum
Log: Desativado
A capturar... usa CTRL+C para parar.
=====

[#1  ] [2026-05-06 14:39:41] Len:72  TTL:128 | Ethernet/IPv4/UDP
.30.223.255:57621
[#2  ] [2026-05-06 14:39:45] Len:73  TTL:255 | Ethernet/IPv4/UDP/DNS
ect._tcp.local: 172.30.208.1:5353 → 224.0.0.251:5353

===== RESUMO ESTATÍSTICO DA CAPTURA =====
Interface: eth0
Total de pacotes validados e processados: 2
Tempo de captura: 0m 15s
```

Figura 3 - Filtro com count 2

```
=====
jordan@PCTiago:~/UMinho/RC/REDES_SNIFFER$ sudo python3 sniffer.py --count 2
===== Packet Sniffer RC-TP2 =====
Interface: eth0
Filtro BPF: Nenhum
Filtro protocolo: Nenhum
Filtro IP: Nenhum
Filtro MAC: Nenhum
Log: Desativado
A capturar... usa CTRL+C para parar.
=====

^C
===== RESUMO ESTATÍSTICO DA CAPTURA =====
Interface: eth0
Total de pacotes validados e processados: 0
Tempo de captura: 0m 0s

Distribuição de Tráfego por Camadas OSI:
=====
```

Figura 4 - Ctrl-c para acabar a captura

3.3. Filtros e Parâmetros

De forma a isolar as comunicações de interesse, a ferramenta implementa uma arquitetura de filtragem híbrida:

- **Filtragem de Baixo Nível:** Filtros **BPF** (Berkeley Packet Filter) que atuam diretamente no **kernel**, rejeitando pacotes irrelevantes antes de atingirem o interpretador Python.
- **Filtragem na Camada de Aplicação:** A função `aplicar_filtros()` executa restrições complexas no código, permitindo isolar tráfego cruzando endereços físicos (MAC), endereços lógicos (IP) e protocolos (ex: filtrar apenas tráfego TCP).

```
===== Packet Sniffer RC-TP2 =====
Interface: eth0
Filtro BPF: Nenhum
Filtro protocolo: HTTP
Filtro IP: Nenhum
Filtro MAC: Nenhum
Log:
capturar... usa CTRL+C para parar.
=====

[#1  ] [2026-05-06 14:59:03] Len:60  TTL:64 | Ethernet/IPv4/TCP/HTTP
04.20.28.246:80 | TCP SYN - início de ligação
[#2  ] [2026-05-06 14:59:03] Len:60  TTL:64 | Ethernet/IPv4/TCP/HTTP
04.20.28.246:80 | TCP SYN - início de ligação

===== RESUMO ESTATÍSTICO DA CAPTURA =====
Interface: eth0
Total de pacotes validados e processados: 2
Tempo de captura: 0m 7s
```

Figura 5 - Captura com filtro HTTP

3.4. Logging de Dados

A ferramenta suporta exportação persistente da auditoria. O formato **CSV** utiliza a classe DictWriter para produzir uma tabela rigorosa contendo os identificadores, hierarquias de encapsulamento e metadados estruturados. O formato **JSON** agrupa os atributos em dicionários aninhados, garantindo a interoperabilidade, e o formato **TXT** preserva o registo visual em texto plano.

```
01 capturajson > ...
02 [
03 {
04   "id_pacote": 1,
05   "ref_pacote": "",
06   "timestamp": "2026-05-06 15:07:43",
07   "interface": "eth0",
08   "protocolo": "ARP",
09   "hierarquia": "Ethernet/ARP",
10   "ttl": "-",
11   "ip_len": "42",
12   "mac_src": "?",
13   "mac_dst": "?",
14   "ip_src": "172.30.208.1",
15   "ip_dst": "172.30.212.63",
16   "portas": "",
17   "tamanho": 42,
18   "resumo": "ARP Request: Quem tem 172.30.212.63? Diz a 172.30.208.1"
19 },
20 {
21   "id_pacote": 2,
22   "ref_pacote": 1,
23   "timestamp": "2026-05-06 15:07:43",
24   "interface": "eth0",
25   "protocolo": "ARP",
26   "hierarquia": "Ethernet/ARP",
27   "ttl": "-",
28   "ip_len": "42",
29   "mac_src": "?",
30   "mac_dst": "?",
31   "ip_src": "172.30.212.63",
32   "ip_dst": "172.30.208.1",
33   "portas": "",
34   "tamanho": 42,
35   "resumo": "ARP Reply: 172.30.212.63 está em 00:15:5d:95:c1:c2 [Resposta ao #1]"
36 }
37 ]
```

Figura 6 - Exportação JSON

3.5. Funcionalidades adicionais

Para complementar a análise estrita de pacotes, a ferramenta integra um módulo de agregação estatística no encerramento da captura (`mostrar_resumo_final()`). O diferencial deste módulo é a sua lógica de distribuição: em vez de enumerar os protocolos numa lista plana, o algoritmo disseca a árvore de encapsulamento de cada pacote e mapeia as contagens diretamente sobre a pilha do Modelo OSI. Recorrendo à estrutura genérica `Counter`, o **sniffer** agrupa o tráfego nas categorias de **Camada 2** (Ligação de Dados), **Camada 3** (Rede), **Camada 4** (Transporte) e **Camada 7** (Aplicação), calculando dinamicamente as percentagens de prevalência da infraestrutura.

```
===== RESUMO ESTATÍSTICO DA CAPTURA =====
Interface: eth0
Total de pacotes validados e processados: 41
Tempo de captura: 2m 43s

Distribuição de Tráfego por Camadas OSI:

[Camada 2 (Ligação de Dados)]
> Ethernet: 41 (100.0%)

[Camada 3 (Rede)]
> IPv4 : 23 (56.1%)
> ARP : 16 (39.0%)
> IPv6 : 2 (4.9%)

[Camada 4 (Transporte)]
> UDP : 25 (61.0%)

[Camada 7 (Aplicação)]
> DNS : 4 (9.8%)

=====
```

Figura 7 - Resumo final com as várias camadas

3.6. Rastreamento de Estado (State Tracking) e Memória de Correlação

O sistema abandona o paradigma da escuta sem estado (stateless). Durante a execução, a aplicação mantém em memória um dicionário dinâmico (memoria_pedidos) que atua como uma tabela de estados heurística. Sempre que interceta um pacote de requisição (ex: ARP Request, DNS Query), atribui-lhe um **ID unívoco incremental** e guarda os metadados. Quando deteta a respetiva resposta, o algoritmo percorre a tabela, reconhece a transação pendente através de chaves únicas e anota a resposta com a referência (ex: [Resposta ao #1]), correlacionando instantaneamente causa e efeito.

```
[#7 ] [2026-05-06 15:22:53] Len:42  TTL:-   | Ethernet/ARP
ARP Request: Quem tem 172.30.212.63? Diz a 172.30.208.1
[#8 ] [2026-05-06 15:22:53] Len:42  TTL:-   | Ethernet/ARP
ARP Reply: 172.30.212.63 está em 00:15:5d:95:c1:c2 [Resposta ao #7]
```

Figura 8 - Exemplo de Rastreamento e memória de correlação

4. Protocolos Analisados

A lógica de dissecção dos pacotes processa-se através de um **pipeline** bottom-up na função identificar_pacote(), espelhando o **Modelo OSI**. Utilizando o método haslayer(), o código valida a integridade das camadas inferiores antes de inspecionar as superiores.

4.1. Camada de Ligação de Dados e ARP

A função obter_macs() verifica o cabeçalho Ether para extrair os **MAC Addresses**. Tratando-se do protocolo **ARP** (Address Resolution Protocol), que atua sem cabeçalhos IP convencionais. A ferramenta extrai os endereços lógicos embutidos na mensagem (arp.psrc e arp.pdst) para garantir o preenchimento da coluna IP no terminal. Simultaneamente, inspeciona o campo arp.op, traduzindo valores 1 como pedidos (ARP Request) e 2 como confirmações (ARP Reply), interligando-os via State Tracking.

```
[#7 ] [2026-05-06 15:22:53] Len:42  TTL:-   | Ethernet/ARP
ARP Request: Quem tem 172.30.212.63? Diz a 172.30.208.1
[#8 ] [2026-05-06 15:22:53] Len:42  TTL:-   | Ethernet/ARP
ARP Reply: 172.30.212.63 está em 00:15:5d:95:c1:c2 [Resposta ao #7]
```

Figura 9 - Exemplo ARP

4.2. Camada de Rede e Inspeção de Encapsulamento (IPv4/IPv6)

Na Camada 3, a função `obter_ips()` identifica a presença da versão **IPv4** ou **IPv6**. Para aprofundar a análise forense, a ferramenta extrai parâmetros vitais do cabeçalho de rede: isola o campo **TTL** (Time To Live / Hop Limit), fundamental para auditar a topologia, e o campo **Total Length** (len / plen), revelando a dimensão do datagrama em **bytes**. Complementarmente, mapeia visualmente a árvore de encapsulamento. Através da função `obter_hierarquia()`, constrói em tempo real a cadeia estrutural (ex: Ethernet/IPv4/TCP/HTTP), fornecendo uma visualização instantânea da profundidade do tráfego.

```
[#166 ] [2026-05-06 15:32:55] Len:198  TTL:1   | Ethernet/IPv4/UDP
9.255.255.250:1900
[#167 ] [2026-05-06 15:32:56] Len:72   TTL:128  | Ethernet/IPv4/UDP
.30.223.255:57621
```

Figura 10 - Exemplo IPv4 com hierarquia

4.3. ICMP

O **Internet Control Message Protocol** é avaliado pelo atributo `icmp.type`. A ferramenta traduz pacotes de tipo 8 para Echo Request (gerados por comandos de teste de alcance), e as respectivas respostas como tipo 0 (Echo Reply), correlacionando ambos com base no identificador da transação de ping original guardado em memória.

```
[#10  ] [2026-05-06 15:52:38] Len:84   TTL:64   | Ethernet/IPv4/ICMP
.8.8.8 | ICMP Echo Request: 172.30.212.63 → 8.8.8.8
[#11  ] [2026-05-06 15:52:38] Len:84   TTL:112  | Ethernet/IPv4/ICMP
212.63 | ICMP Echo Reply: 8.8.8.8 → 172.30.212.63 [Resposta ao #10]
```

Figura 11 - Exemplo ICMP com respostas

4.4. Transporte (TCP) e Inspeção Aplicacional (HTTP/HTTPS)

O processamento **TCP** inclui a abstração da máquina de estados do protocolo. Acedendo às flags via `sprintf("%flags%")`, cataloga as sessões ativas: SYN para início de handshake, SYN-ACK para anuência, e FIN ou RST para término. Na Camada de Aplicação, a deteção da porta 80 despoleta a `obter_http_info()`. O código acede à carga útil nua (`pacote[Raw].load`), converte os bytes em string (UTF-8) e extrai os métodos HTTP (GET, POST) diretamente do datagrama, executando verdadeira Inspeção Profunda de Pacotes. O tráfego na porta 443 é corretamente assinalado como HTTPS (cifrado).

```
[#9  ] [2026-05-06 17:32:48] Len:92  TTL:117 | Ethernet/IPv4/TCP/HTTPS  | MAC: d4:7f:35:16:e2:9f → d0:39:57:19:4a:a5 | IP: 35.186.224.33 → 172.26.204.219 | HTTPS: 35.186.224.33:443 → 172.26.204.219:57478 | tráfego cifrado | TCP PSH-ACK - envio de dados
[#10 ] [2026-05-06 17:32:48] Len:52  TTL:64 | Ethernet/IPv4/TCP/HTTPS  | MAC: d0:39:57:19:4a:a5 → d4:7f:35:16:e2:9f | IP: 172.26.204.219 → 35.186.224.33 | HTTPS: 172.26.204.219:57478 → 35.186.224.33:443 | tráfego cifrado | TCP ACK - confirmação
```

Figura 12 - Exemplo HTTPS

4.5. UDP e Dissecção de DNS

Tratando-se do protocolo de datagramas **UDP**, o foco recai sobre a resolução de nomes de domínio (porta 53). A dissecção do protocolo DNS avalia a direção em `dns.qr`. Nas submissões (`qr=0`), a query contida em **DNSQR** é extraída e decodificada. Nas respostas (`qr=1`), o código itera sobre o registo **DNSRR**, capturando e listando os IPs resolvidos pelo servidor de forma a providenciar **contexto legível** ao administrador, e rastreia o pacote de volta ao pedido original de DNS.

```
[#1  ] [2026-05-06 16:01:45] Len:73  TTL:255 | Ethernet/IPv4/UDP/DNS  | MAC: 00:0c:29:14:92:34 → 172.30.208.1:5353 → 224.0.0.251:5353 | DNS Query: domínio=_spotify-connect._tcp.local: 172.30.208.1:5353 → 224.0.0.251:5353
[#2  ] [2026-05-06 16:01:45] Len:53  TTL:255 | Ethernet/IPv6/UDP/DNS  | MAC: 00:02::fb | DNS Query: domínio=_spotify-connect._tcp.local: fe80::3902:fad:3b5a:26fc:5353 → 224.0.0.251:5353
[#3  ] [2026-05-06 16:03:44] Len:73  TTL:255 | Ethernet/IPv4/UDP/DNS  | MAC: 00:0c:29:14:92:34 → 172.30.208.1:5353 → 224.0.0.251:5353 | DNS Query: domínio=_spotify-connect._tcp.local: 172.30.208.1:5353 → 224.0.0.251:5353
```

Figura 13 - Exemplo de DNS que capta spotify

5. Demonstração e Teste no CORE (Parte A)

Para validar o funcionamento da aplicação, realizamos testes de captura no emulador de redes CORE. A principal diferença e grande vantagem de testar neste ambiente virtual é o isolamento absoluto do meio de comunicação, impossível de obter no mundo real. Numa placa de rede real, o sniffer seria imediatamente inundado por pacotes de diversos protocolos. Assim, o CORE permite-nos criar cenários controlados e repetíveis, onde cada pacote capturado é um resultado direto de uma ação específica que pretendemos observar. A topologia de teste consistiu em dois nós de extremidade (n1 e n2), interligados através de um *router* central (n3), operando em sub-redes distintas. Esta configuração controlada assegurou-nos que cada pacote interceptado resultava exclusivamente de um comando nosso, tornando a validação do processo de desencapsulamento muito mais precisa e previsível.

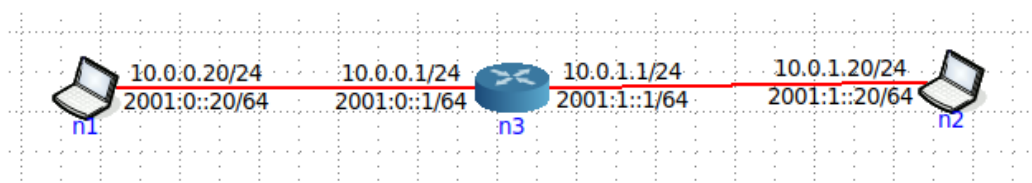


Figura 14 - Topologia de rede isolada no emulador CORE.

O primeiro passo prático consistiu na inicialização do nosso programa no nó recetor (n1). Invocamos a aplicação através do comando **python3 sniffer.py -i eth0 --log captura_core.csv**. A indicação da interface eth0 foi estritamente necessária para focar a escuta na ligação correta, enquanto a flag --log cumpriu o requisito de armazenamento persistente da captura em ficheiro.

O passo seguinte, após configurar o *sniffer* para escuta ativa, centrou-se no nó emissor (n2), onde geramos o tráfego de teste.

A primeira ação de gerar tráfego consistiu na execução do comando **ping 10.0.0.20** (endereço de n1) em n2, o que nos permitiu testar dois protocolos em simultâneo (ARP e ICMP). Como os nós n1 e n2 pertencem a sub-redes diferentes, o n2 não possuía o endereço MAC do n1, mas sim do seu *gateway*, o *router* n3. Ao executar um comando ping, o tráfego teve de passar pelo router. O router, por sua vez, ao encaminhar o pacote para n1, teve de executar um ARP Request para descobrir o MAC do n1. Este pedido e resposta foram capturados pelo *sniffer* em n1, comprovando a correta dissecação do protocolo ARP.

Na sequência desta resolução, o sniffer capturou sequencialmente os pacotes *Echo Request* (n2 para n1) e *Echo Reply* (n1 para n2), validando não só a capacidade de analisar os cabeçalhos IP e ICMP, mas também de distinguir a direção do tráfego.

Por fim, geramos tráfego TCP e UDP para validar a análise da Camada 4. No nó n2, simulamos uma comunicação TCP através do comando **wget http://10.0.0.20**. Como não existia um servidor web a correr em n1, a ligação foi rejeitada. Esta ação demonstrou a capacidade do sniffer de identificar perfeitamente as diferentes flags TCP associadas, capturando o pedido de sincronização (SYN) e a imediata rejeição forçada (RST). Para o UDP, usamos o comando **echo "Ola Sniffer" | nc -u 10.0.0.20 5555** enviando uma mensagem simples, comprovando a análise correta dos datagramas UDP num serviço local.

Todos os pacotes capturados e as suas respetivas informações (Camadas, Endereços MAC/IP, Portas, e *Payload* relevante) foram registados no ficheiro `captura_core.csv`, conforme solicitado. A análise deste ficheiro confirmou que o desencapsulamento e a extração de dados estavam a funcionar de forma precisa e robusta em ambientes isolados.

```
root@n2:/tmp/pycore.44261/n2.conf# ping -c 4 10.0.0.20
PING 10.0.0.20 (10.0.0.20) 56(84) bytes of data:
64 bytes from 10.0.0.20: icmp_seq=1 ttl=63 time=0.166 ms
64 bytes from 10.0.0.20: icmp_seq=2 ttl=63 time=0.072 ms
64 bytes from 10.0.0.20: icmp_seq=3 ttl=63 time=0.071 ms
64 bytes from 10.0.0.20: icmp_seq=4 ttl=63 time=0.101 ms

--- 10.0.0.20 ping statistics ---
4 packets transmitted, 4 received, 0% packet loss, time 3034ms
rtt min/avg/max/mdev = 0.071/0.102/0.166/0.038 ms
root@n2:/tmp/pycore.44261/n2.conf# wget http://10.0.0.20
--2026-05-06 00:28:22-- http://10.0.0.20/
Connecting to 10.0.0.20:80... failed: Connection refused.
root@n2:/tmp/pycore.44261/n2.conf# echo "Ola Sniffer" | nc -u 10.0.0.20 5555
root@n2:/tmp/pycore.44261/n2.conf#
```

Figura 15 - Execução de comandos no nó emissor (n2) para gerar o tráfego de teste.

```

root@n1:/home/core/REIES.Sniffer# python3 sniffer.py -i eth0 --log captura_core.csv
Interface: eth0
Filtro BPF: Nenhum
Filtro protocolo: Nenhum
Filtro IP: Nenhum
Filtro MAC: Nenhum
Log: captura_core.csv
A capturar... usa CTRL+C para parar.

=====
#1 | [2026-05-06 17:38:07] Len164 | TTL:1 | Ethernet/IPv4 | MAC: 00:00:00:aa:00:01 + 01:00:5e:00:00:05 | IP: 10.0.0.1 + 224.0.0.5 | IPv4: 10.0.0.1 + 224.0.0.5
#2 | [2026-05-06 17:38:08] Len136 | TTL:1 | Ethernet/IPv6 | MAC: 00:00:00:aa:00:01 + 33:33:00:00:00:05 | IP: fe80::200:ff:feaa:1 + ff02::5 | IPv6: fe80::200:ff:feaa:1 + ff02::5
#3 | [2026-05-06 17:38:08] Len164 | TTL:1 | Ethernet/IPv4 | MAC: 00:00:00:aa:00:01 + 01:00:5e:00:00:05 | IP: 10.0.0.1 + 224.0.0.5 | IPv4: 10.0.0.1 + 224.0.0.5
#4 | [2026-05-06 17:38:09] Len184 | TTL:63 | Ethernet/IPv4/ICMP | MAC: 00:00:00:aa:00:01 + 00:00:00:aa:00:00 | IP: 10.0.1.20 + 10.0.0.20 | ICMP Echo Request: 10.0.1.20 + 10.0.0.20
#5 | [2026-05-06 17:38:09] Len184 | TTL:64 | Ethernet/IPv4/ICMP | MAC: 00:00:00:aa:00:00 + 00:00:00:aa:00:01 | IP: 10.0.0.20 + 10.0.1.20 | ICMP Echo Reply: 10.0.0.20 + 10.0.1.20 [Resposta ao #4]
#6 | [2026-05-06 17:38:10] Len184 | TTL:63 | Ethernet/IPv4/ICMP | MAC: 00:00:00:aa:00:01 + 00:00:00:aa:00:00 | IP: 10.0.1.20 + 10.0.0.20 | ICMP Echo Request: 10.0.1.20 + 10.0.0.20
#7 | [2026-05-06 17:38:10] Len184 | TTL:64 | Ethernet/IPv4/ICMP | MAC: 00:00:00:aa:00:00 + 00:00:00:aa:00:01 | IP: 10.0.0.20 + 10.0.1.20 | ICMP Echo Reply: 10.0.0.20 + 10.0.1.20 [Resposta ao #6]
#8 | [2026-05-06 17:38:11] Len164 | TTL:1 | Ethernet/IPv4 | MAC: 00:00:00:aa:00:01 + 01:00:5e:00:00:05 | IP: 10.0.0.1 + 224.0.0.5 | IPv4: 10.0.0.1 + 224.0.0.5
#9 | [2026-05-06 17:38:11] Len184 | TTL:63 | Ethernet/IPv4/ICMP | MAC: 00:00:00:aa:00:01 + 00:00:00:aa:00:00 | IP: 10.0.1.20 + 10.0.0.20 | ICMP Echo Request: 10.0.1.20 + 10.0.0.20
#10 | [2026-05-06 17:38:11] Len184 | TTL:64 | Ethernet/IPv4/ICMP | MAC: 00:00:00:aa:00:00 + 00:00:00:aa:00:01 | IP: 10.0.0.20 + 10.0.1.20 | ICMP Echo Reply: 10.0.0.20 + 10.0.1.20 [Resposta ao #9]
#11 | [2026-05-06 17:38:12] Len184 | TTL:63 | Ethernet/IPv4/ICMP | MAC: 00:00:00:aa:00:01 + 00:00:00:aa:00:00 | IP: 10.0.1.20 + 10.0.0.20 | ICMP Echo Request: 10.0.1.20 + 10.0.0.20
#12 | [2026-05-06 17:38:12] Len184 | TTL:64 | Ethernet/IPv4/ICMP | MAC: 00:00:00:aa:00:00 + 00:00:00:aa:00:01 | IP: 10.0.0.20 + 10.0.1.20 | ICMP Echo Reply: 10.0.0.20 + 10.0.1.20 [Resposta ao #11]
#13 | [2026-05-06 17:38:13] Len164 | TTL:1 | Ethernet/IPv4 | MAC: 00:00:00:aa:00:01 + 01:00:5e:00:00:05 | IP: 10.0.0.1 + 224.0.0.5 | IPv4: 10.0.0.1 + 224.0.0.5
#14 | [2026-05-06 17:38:14] Len160 | TTL:63 | Ethernet/IPv4/TCP/HTTP | MAC: 00:00:00:aa:00:01 + 00:00:00:aa:00:00 | IP: 10.0.1.20 + 10.0.0.20 | HTTP: 10.0.1.20:59288 + 10.0.0.20:80 | TCP SYN - início de ligação
#15 | [2026-05-06 17:38:14] Len140 | TTL:64 | Ethernet/IPv4/TCP/HTTP | MAC: 00:00:00:aa:00:00 + 00:00:00:aa:00:01 | IP: 10.0.0.20 + 10.0.1.20 | HTTP: 10.0.0.20:80 + 10.0.1.20:59288 | TCP RST - ligação reiniciada
#16 | [2026-05-06 17:38:14] Len142 | TTL:- | Ethernet/ARP | MAC: ? + ? | IP: 10.0.0.20 + 10.0.0.1 | ARP Request: Quem tem 10.0.0.1? Diz a 10.0.0.20
#17 | [2026-05-06 17:38:14] Len142 | TTL:- | Ethernet/ARP | MAC: ? + ? | IP: 10.0.0.1 + 10.0.0.20 | ARP Request: Quem tem 10.0.0.20? Diz a 10.0.0.1
#18 | [2026-05-06 17:38:14] Len142 | TTL:- | Ethernet/ARP | MAC: ? + ? | IP: 10.0.0.20 + 10.0.0.1 | ARP Reply: 10.0.0.20 está em 00:00:00:aa:00:00 [Resposta ao #17]
#19 | [2026-05-06 17:38:14] Len142 | TTL:- | Ethernet/ARP | MAC: ? + ? | IP: 10.0.0.1 + 10.0.0.20 | ARP Reply: 10.0.0.1 está em 00:00:00:aa:00:01 [Resposta ao #16]
#20 | [2026-05-06 17:38:15] Len164 | TTL:1 | Ethernet/IPv4 | MAC: 00:00:00:aa:00:01 + 01:00:5e:00:00:05 | IP: 10.0.0.1 + 224.0.0.5 | IPv4: 10.0.0.1 + 224.0.0.5
#21 | [2026-05-06 17:38:15] Len140 | TTL:63 | Ethernet/IPv4/UDP | MAC: 00:00:00:aa:00:01 + 00:00:00:aa:00:00 | IP: 10.0.1.20 + 10.0.0.20 | UDP: 10.0.1.20:34802 + 10.0.0.20:5555
#22 | [2026-05-06 17:38:15] Len168 | TTL:64 | Ethernet/IPv4/ICMP | MAC: 00:00:00:aa:00:00 + 00:00:00:aa:00:01 | IP: 10.0.0.20 + 10.0.1.20 | ICMP tipo3: 10.0.0.20 + 10.0.1.20
#23 | [2026-05-06 17:38:17] Len164 | TTL:1 | Ethernet/IPv4 | MAC: 00:00:00:aa:00:01 + 01:00:5e:00:00:05 | IP: 10.0.0.1 + 224.0.0.5 | IPv4: 10.0.0.1 + 224.0.0.5
#24 | [2026-05-06 17:38:18] Len136 | TTL:1 | Ethernet/IPv6 | MAC: 00:00:00:aa:00:01 + 33:33:00:00:00:05 | IP: fe80::200:ff:feaa:1 + ff02::5 | IPv6: fe80::200:ff:feaa:1 + ff02::5
#25 | [2026-05-06 17:38:19] Len164 | TTL:1 | Ethernet/IPv4 | MAC: 00:00:00:aa:00:01 + 01:00:5e:00:00:05 | IP: 10.0.0.1 + 224.0.0.5 | IPv4: 10.0.0.1 + 224.0.0.5
#26 | [2026-05-06 17:38:21] Len164 | TTL:1 | Ethernet/IPv4 | MAC: 00:00:00:aa:00:01 + 01:00:5e:00:00:05 | IP: 10.0.0.1 + 224.0.0.5 | IPv4: 10.0.0.1 + 224.0.0.5
#27 | [2026-05-06 17:38:23] Len164 | TTL:1 | Ethernet/IPv4 | MAC: 00:00:00:aa:00:01 + 01:00:5e:00:00:05 | IP: 10.0.0.1 + 224.0.0.5 | IPv4: 10.0.0.1 + 224.0.0.5
#28 | [2026-05-06 17:38:25] Len164 | TTL:1 | Ethernet/IPv4 | MAC: 00:00:00:aa:00:01 + 01:00:5e:00:00:05 | IP: 10.0.0.1 + 224.0.0.5 | IPv4: 10.0.0.1 + 224.0.0.5
#29 | [2026-05-06 17:38:27] Len164 | TTL:1 | Ethernet/IPv4 | MAC: 00:00:00:aa:00:01 + 01:00:5e:00:00:05 | IP: 10.0.0.1 + 224.0.0.5 | IPv4: 10.0.0.1 + 224.0.0.5
#30 | [2026-05-06 17:38:28] Len136 | TTL:1 | Ethernet/IPv6 | MAC: 00:00:00:aa:00:01 + 33:33:00:00:00:05 | IP: fe80::200:ff:feaa:1 + ff02::5 | IPv6: fe80::200:ff:feaa:1 + ff02::5
#31 | [2026-05-06 17:38:29] Len164 | TTL:1 | Ethernet/IPv4 | MAC: 00:00:00:aa:00:01 + 01:00:5e:00:00:05 | IP: 10.0.0.1 + 224.0.0.5 | IPv4: 10.0.0.1 + 224.0.0.5

=====
RESUMO ESTATÍSTICO DA CAPTURA
Interface: eth0
Total de pacotes validados e processados: 31
Tempo de captura: 0m 25s

Distribuição de Tráfego por Camadas OSI:

[Camada 2 (Ligação de Dados)]
> Ethernet: 31 (100.0%)

[Camada 3 (Rede)]
> IPv4 : 24 (77.4%)
> ICMP : 3 (9.7%)
> ARP : 4 (12.9%)
> IPv6 : 3 (9.7%)

[Camada 4 (Transporte)]
> TCP : 2 (6.5%)
> UDP : 1 (3.2%)

[Camada 7 (Aplicação)]
> HTTP : 2 (6.5%)

```

Figura 16 - Captura, análise de pacotes em tempo real e resumo estatístico no nó recetor (n1).

6. Demonstração na Interface Real (Parte B)

Esta fase teve como objetivo observar o comportamento do sniffer perante tráfego real gerado pelo sistema operativo e pelas aplicações utilizadas diariamente, permitindo validar os mecanismos de captura, filtragem, identificação de protocolos e exportação de dados implementados.

6.1. Execução do Sniffer na Interface Real

Nesta fase, o packet sniffer foi executado diretamente na interface real do computador, utilizando a interface Wi-Fi wlo1. O objetivo principal desta etapa foi validar o funcionamento da aplicação fora do ambiente controlado do CORE, permitindo observar tráfego real gerado pelo sistema operativo e pelas aplicações utilizadas no computador.

A captura foi realizada com permissões de administrador (sudo), uma vez que a captura de pacotes numa interface física requer acesso privilegiado ao sistema operativo. Sem estas permissões, o programa não consegue colocar a interface em modo de escuta nem intercetar corretamente os pacotes que circulam na rede.

Ao iniciar a aplicação sem argumentos de linha de comandos, o sniffer apresenta automaticamente um menu interativo responsável pela configuração da captura. Através da função `get_if_list()` da biblioteca Scapy, são listadas todas as interfaces de rede disponíveis no sistema, permitindo ao utilizador selecionar a interface pretendida.

Além da escolha da interface, o menu permite configurar:

- número de pacotes a capturar;
- filtros por protocolo;
- filtros BPF;
- filtros por IP e MAC;
- exportação para ficheiro.

```

dremar@83dremar:~/Desktop/universidade/redes/REDES_SNIFFER$ sudo python3 sniffer.py

===== CONFIGURAÇÃO DO SNIFFER =====

Interfaces disponíveis:
0 - lo
1 - eno1
2 - wlo1

Escolhe a interface pelo número: 2

Quantos pacotes queres capturar? [0 para infinito]: 15

Protocolo a filtrar (ex: ARP, ICMP, TCP, DNS) [Enter para todos]: icmp

Filtro BPF (ex: tcp, udp, port 53) [Enter para nenhum]:

Filtrar por IP [Enter para nenhum]:

Filtrar por MAC [Enter para nenhum]:

Guardar log? [s/n]: n

===== CONFIGURAÇÃO FINAL =====
Interface: wlo1
Pacotes: 15
Protocolo: icmp
Filtro BPF: Nenhum
IP: Nenhum
MAC: Nenhum
Log: Desativado
=====

===== Packet Sniffer RC-TP2 =====
Interface: wlo1
Filtro BPF: Nenhum
Filtro protocolo: icmp
Filtro IP: Nenhum
Filtro MAC: Nenhum
Log: Desativado
A capturar... usa CTRL+C para parar.
=====

```

Figura 17 - Configuração inicial do packet sniffer na interface real wlo1.

Após a configuração, a aplicação inicia imediatamente a monitorização da interface seleccionada, analisando os pacotes recebidos em tempo real.

A nova estrutura da saída apresentada no terminal inclui agora informação mais detalhada sobre cada pacote capturado, nomeadamente:

- identificador do pacote;
- timestamp;
- tamanho do pacote IP;
- valor TTL;
- hierarquia de encapsulamento;
- endereços MAC;
- endereços IP;
- resumo interpretado da comunicação.

Esta informação permitiu obter uma visualização mais completa do tráfego capturado e compreender melhor o processo de desencapsulamento das diferentes camadas do modelo OSI.

```

===== Packet Sniffer RC-TP2 =====
Interface: wlan1
Filtro BPF: Nenhum
Filtro protocolo: icmp
Filtro IP: Nenhum
Filtro MAC: Nenhum
Log: Desativado
A capturar... usa CTRL+C para parar.
=====

[#1 ] [2026-05-06 17:23:02] Len:84  TTL:64  | Ethernet/IPv4/ICMP  | MAC: a8:41:f4:e6:e3:6d → d4:7f:35:16:e2:9f | IP: 172.26.29.86 → 8.8.8.8 | ICMP Echo Request: 172.26.29.86
→ 8.8.8.8
[#2 ] [2026-05-06 17:23:02] Len:84  TTL:113 | Ethernet/IPv4/ICMP  | MAC: d4:7f:35:16:e2:9f → a8:41:f4:e6:e3:6d | IP: 8.8.8.8 → 172.26.29.86 | ICMP Echo Reply: 8.8.8.8 → 172.2
6.29.86
[#3 ] [2026-05-06 17:23:03] Len:84  TTL:64  | Ethernet/IPv4/ICMP  | MAC: a8:41:f4:e6:e3:6d → d4:7f:35:16:e2:9f | IP: 172.26.29.86 → 8.8.8.8 | ICMP Echo Request: 172.26.29.86
→ 8.8.8.8
[#4 ] [2026-05-06 17:23:03] Len:84  TTL:113 | Ethernet/IPv4/ICMP  | MAC: d4:7f:35:16:e2:9f → a8:41:f4:e6:e3:6d | IP: 8.8.8.8 → 172.26.29.86 | ICMP Echo Reply: 8.8.8.8 → 172.2
6.29.86
[#5 ] [2026-05-06 17:23:04] Len:84  TTL:64  | Ethernet/IPv4/ICMP  | MAC: a8:41:f4:e6:e3:6d → d4:7f:35:16:e2:9f | IP: 172.26.29.86 → 8.8.8.8 | ICMP Echo Request: 172.26.29.86
→ 8.8.8.8
[#6 ] [2026-05-06 17:23:04] Len:84  TTL:113 | Ethernet/IPv4/ICMP  | MAC: d4:7f:35:16:e2:9f → a8:41:f4:e6:e3:6d | IP: 8.8.8.8 → 172.26.29.86 | ICMP Echo Reply: 8.8.8.8 → 172.2
6.29.86
[#7 ] [2026-05-06 17:23:05] Len:84  TTL:64  | Ethernet/IPv4/ICMP  | MAC: a8:41:f4:e6:e3:6d → d4:7f:35:16:e2:9f | IP: 172.26.29.86 → 8.8.8.8 | ICMP Echo Request: 172.26.29.86
→ 8.8.8.8
[#8 ] [2026-05-06 17:23:05] Len:84  TTL:113 | Ethernet/IPv4/ICMP  | MAC: d4:7f:35:16:e2:9f → a8:41:f4:e6:e3:6d | IP: 8.8.8.8 → 172.26.29.86 | ICMP Echo Reply: 8.8.8.8 → 172.2
6.29.86
[#9 ] [2026-05-06 17:23:06] Len:84  TTL:64  | Ethernet/IPv4/ICMP  | MAC: a8:41:f4:e6:e3:6d → d4:7f:35:16:e2:9f | IP: 172.26.29.86 → 8.8.8.8 | ICMP Echo Request: 172.26.29.86
→ 8.8.8.8
[#10 ] [2026-05-06 17:23:06] Len:84  TTL:113 | Ethernet/IPv4/ICMP  | MAC: d4:7f:35:16:e2:9f → a8:41:f4:e6:e3:6d | IP: 8.8.8.8 → 172.26.29.86 | ICMP Echo Reply: 8.8.8.8 → 172.2
6.29.86
[#11 ] [2026-05-06 17:23:07] Len:84  TTL:64  | Ethernet/IPv4/ICMP  | MAC: a8:41:f4:e6:e3:6d → d4:7f:35:16:e2:9f | IP: 172.26.29.86 → 8.8.8.8 | ICMP Echo Request: 172.26.29.86
→ 8.8.8.8
[#12 ] [2026-05-06 17:23:07] Len:84  TTL:113 | Ethernet/IPv4/ICMP  | MAC: d4:7f:35:16:e2:9f → a8:41:f4:e6:e3:6d | IP: 8.8.8.8 → 172.26.29.86 | ICMP Echo Reply: 8.8.8.8 → 172.2
6.29.86
[#13 ] [2026-05-06 17:23:08] Len:84  TTL:64  | Ethernet/IPv4/ICMP  | MAC: a8:41:f4:e6:e3:6d → d4:7f:35:16:e2:9f | IP: 172.26.29.86 → 8.8.8.8 | ICMP Echo Request: 172.26.29.86
→ 8.8.8.8
[#14 ] [2026-05-06 17:23:08] Len:84  TTL:113 | Ethernet/IPv4/ICMP  | MAC: d4:7f:35:16:e2:9f → a8:41:f4:e6:e3:6d | IP: 8.8.8.8 → 172.26.29.86 | ICMP Echo Reply: 8.8.8.8 → 172.2
6.29.86
[#15 ] [2026-05-06 17:23:09] Len:84  TTL:64  | Ethernet/IPv4/ICMP  | MAC: a8:41:f4:e6:e3:6d → d4:7f:35:16:e2:9f | IP: 172.26.29.86 → 8.8.8.8 | ICMP Echo Request: 172.26.29.86
→ 8.8.8.8

===== RESUMO ESTATÍSTICO DA CAPTURA =====
Interface: wlan1
Total de pacotes validados e processados: 15
Tempo de captura: 0m 15s

```

Figura 18 - Início da captura de pacotes na interface real.

6.2. Captura e Observação de Tráfego Real

Após a inicialização do sniffer na interface wlo1, foram realizados vários testes para gerar tráfego real e validar a capacidade da aplicação em identificar diferentes protocolos presentes na rede.

Durante a captura, o sniffer apresentou informação detalhada sobre cada pacote observado, incluindo:

- identificador do pacote;
- tamanho IP;
- valor TTL;
- hierarquia de encapsulamento;
- endereços MAC;
- endereços IP;
- resumo da comunicação.

A identificação dos protocolos foi realizada através da função `haslayer()` da biblioteca Scapy, permitindo analisar sequencialmente as diferentes camadas do pacote.

```
===== Packet Sniffer RC-TP2 =====
Interface: wlo1
Filtro BPF: Nenhum
Filtro protocolo: icmp
Filtro IP: Nenhum
Filtro MAC: Nenhum
Log: Desativado
A capturar... usa CTRL+C para parar.
=====

[#1 ] [2026-05-06 20:18:19] Len:84  TTL:64 | Ethernet/IPv4/ICMP | MAC: a8:41:f4:e6:e3:6d → 14:ca:56:4d:ff:3b | IP: 192.168.1.218 → 192.168.0.0 | ICMP
Echo Request: 192.168.1.218 → 192.168.0.0
[#2 ] [2026-05-06 20:18:20] Len:84  TTL:64 | Ethernet/IPv4/ICMP | MAC: a8:41:f4:e6:e3:6d → 14:ca:56:4d:ff:3b | IP: 192.168.1.218 → 192.168.0.0 | ICMP
Echo Request: 192.168.1.218 → 192.168.0.0
[#3 ] [2026-05-06 20:18:21] Len:84  TTL:64 | Ethernet/IPv4/ICMP | MAC: a8:41:f4:e6:e3:6d → 14:ca:56:4d:ff:3b | IP: 192.168.1.218 → 192.168.0.0 | ICMP
Echo Request: 192.168.1.218 → 192.168.0.0
[#4 ] [2026-05-06 20:18:22] Len:84  TTL:64 | Ethernet/IPv4/ICMP | MAC: a8:41:f4:e6:e3:6d → 14:ca:56:4d:ff:3b | IP: 192.168.1.218 → 192.168.0.0 | ICMP
Echo Request: 192.168.1.218 → 192.168.0.0
[#5 ] [2026-05-06 20:18:30] Len:84  TTL:64 | Ethernet/IPv4/ICMP | MAC: a8:41:f4:e6:e3:6d → 14:ca:56:4d:ff:3b | IP: 192.168.1.218 → 8.8.8.8 | ICMP Echo
Request: 192.168.1.218 → 8.8.8.8
[#6 ] [2026-05-06 20:18:30] Len:84  TTL:116 | Ethernet/IPv4/ICMP | MAC: 14:ca:56:4d:ff:3b → a8:41:f4:e6:e3:6d | IP: 8.8.8.8 → 192.168.1.218 | ICMP Echo
Reply: 8.8.8.8 → 192.168.1.218
[#7 ] [2026-05-06 20:18:31] Len:84  TTL:64 | Ethernet/IPv4/ICMP | MAC: a8:41:f4:e6:e3:6d → 14:ca:56:4d:ff:3b | IP: 192.168.1.218 → 8.8.8.8 | ICMP Echo
Request: 192.168.1.218 → 8.8.8.8
[#8 ] [2026-05-06 20:18:31] Len:84  TTL:116 | Ethernet/IPv4/ICMP | MAC: 14:ca:56:4d:ff:3b → a8:41:f4:e6:e3:6d | IP: 8.8.8.8 → 192.168.1.218 | ICMP Echo
Reply: 8.8.8.8 → 192.168.1.218
[#9 ] [2026-05-06 20:18:32] Len:84  TTL:64 | Ethernet/IPv4/ICMP | MAC: a8:41:f4:e6:e3:6d → 14:ca:56:4d:ff:3b | IP: 192.168.1.218 → 8.8.8.8 | ICMP Echo
Request: 192.168.1.218 → 8.8.8.8
[#10] [2026-05-06 20:18:32] Len:84  TTL:116 | Ethernet/IPv4/ICMP | MAC: 14:ca:56:4d:ff:3b → a8:41:f4:e6:e3:6d | IP: 8.8.8.8 → 192.168.1.218 | ICMP Echo
Reply: 8.8.8.8 → 192.168.1.218

===== RESUMO ESTATÍSTICO DA CAPTURA =====
Interface: wlo1
Total de pacotes validados e processados: 10
Tempo de captura: 0m 13s

Distribuição de Tráfego por Camadas OSI:

[Camada 2 (Ligação de Dados)]
> Ethernet: 10 (100.0%)

[Camada 3 (Rede)]
> IPv4 : 10 (100.0%)
> ICMP : 10 (100.0%)

=====
```

Figura 19 - Captura de pacotes em tempo real na interface Wi-Fi.

6.2.1 Captura de tráfego ICMP

Para gerar tráfego ICMP foi utilizado o comando: **ping 8.8.8.8**

Durante a captura, o sniffer identificou corretamente mensagens ICMP Echo Request e ICMP Echo Reply.

Além disso, através do sistema de rastreamento implementado, algumas respostas puderam ser associadas aos respectivos pedidos originais.

```
===== Packet Sniffer RC-TP2 =====
Interface: wlo1
Filtro BPF: Nenhum
Filtro protocolo: icmp
Filtro IP: Nenhum
Filtro MAC: Nenhum
Log: Desativado
A capturar... usa CTRL+C para parar.
=====

[ #1 ] [2026-05-06 17:23:02] Len:84   TTL:64 | Ethernet/IPv4/ICMP | MAC: a8:41:f4:e6:e3:6d → d4:7f:35:16:e2:9f | IP: 172.26.29.86 → 8.8.8.8 | ICMP
Echo Request: 172.26.29.86 → 8.8.8.8
[ #2 ] [2026-05-06 17:23:02] Len:84   TTL:113 | Ethernet/IPv4/ICMP | MAC: d4:7f:35:16:e2:9f → a8:41:f4:e6:e3:6d | IP: 8.8.8.8 → 172.26.29.86 | ICMP
Echo Reply: 8.8.8.8 → 172.26.29.86
[ #3 ] [2026-05-06 17:23:03] Len:84   TTL:64 | Ethernet/IPv4/ICMP | MAC: a8:41:f4:e6:e3:6d → d4:7f:35:16:e2:9f | IP: 172.26.29.86 → 8.8.8.8 | ICMP
Echo Request: 172.26.29.86 → 8.8.8.8
[ #4 ] [2026-05-06 17:23:03] Len:84   TTL:113 | Ethernet/IPv4/ICMP | MAC: d4:7f:35:16:e2:9f → a8:41:f4:e6:e3:6d | IP: 8.8.8.8 → 172.26.29.86 | ICMP
Echo Reply: 8.8.8.8 → 172.26.29.86
[ #5 ] [2026-05-06 17:23:04] Len:84   TTL:64 | Ethernet/IPv4/ICMP | MAC: a8:41:f4:e6:e3:6d → d4:7f:35:16:e2:9f | IP: 172.26.29.86 → 8.8.8.8 | ICMP
Echo Request: 172.26.29.86 → 8.8.8.8
[ #6 ] [2026-05-06 17:23:04] Len:84   TTL:113 | Ethernet/IPv4/ICMP | MAC: d4:7f:35:16:e2:9f → a8:41:f4:e6:e3:6d | IP: 8.8.8.8 → 172.26.29.86 | ICMP
Echo Reply: 8.8.8.8 → 172.26.29.86
[ #7 ] [2026-05-06 17:23:05] Len:84   TTL:64 | Ethernet/IPv4/ICMP | MAC: a8:41:f4:e6:e3:6d → d4:7f:35:16:e2:9f | IP: 172.26.29.86 → 8.8.8.8 | ICMP
Echo Request: 172.26.29.86 → 8.8.8.8
[ #8 ] [2026-05-06 17:23:05] Len:84   TTL:113 | Ethernet/IPv4/ICMP | MAC: d4:7f:35:16:e2:9f → a8:41:f4:e6:e3:6d | IP: 8.8.8.8 → 172.26.29.86 | ICMP
Echo Reply: 8.8.8.8 → 172.26.29.86
[ #9 ] [2026-05-06 17:23:06] Len:84   TTL:64 | Ethernet/IPv4/ICMP | MAC: a8:41:f4:e6:e3:6d → d4:7f:35:16:e2:9f | IP: 172.26.29.86 → 8.8.8.8 | ICMP
Echo Request: 172.26.29.86 → 8.8.8.8
[ #10 ] [2026-05-06 17:23:06] Len:84   TTL:113 | Ethernet/IPv4/ICMP | MAC: d4:7f:35:16:e2:9f → a8:41:f4:e6:e3:6d | IP: 8.8.8.8 → 172.26.29.86 | ICMP
Echo Reply: 8.8.8.8 → 172.26.29.86
[ #11 ] [2026-05-06 17:23:07] Len:84   TTL:64 | Ethernet/IPv4/ICMP | MAC: a8:41:f4:e6:e3:6d → d4:7f:35:16:e2:9f | IP: 172.26.29.86 → 8.8.8.8 | ICMP
Echo Request: 172.26.29.86 → 8.8.8.8
[ #12 ] [2026-05-06 17:23:07] Len:84   TTL:113 | Ethernet/IPv4/ICMP | MAC: d4:7f:35:16:e2:9f → a8:41:f4:e6:e3:6d | IP: 8.8.8.8 → 172.26.29.86 | ICMP
Echo Reply: 8.8.8.8 → 172.26.29.86
[ #13 ] [2026-05-06 17:23:08] Len:84   TTL:64 | Ethernet/IPv4/ICMP | MAC: a8:41:f4:e6:e3:6d → d4:7f:35:16:e2:9f | IP: 172.26.29.86 → 8.8.8.8 | ICMP
Echo Request: 172.26.29.86 → 8.8.8.8
[ #14 ] [2026-05-06 17:23:08] Len:84   TTL:113 | Ethernet/IPv4/ICMP | MAC: d4:7f:35:16:e2:9f → a8:41:f4:e6:e3:6d | IP: 8.8.8.8 → 172.26.29.86 | ICMP
Echo Reply: 8.8.8.8 → 172.26.29.86
[ #15 ] [2026-05-06 17:23:09] Len:84   TTL:64 | Ethernet/IPv4/ICMP | MAC: a8:41:f4:e6:e3:6d → d4:7f:35:16:e2:9f | IP: 172.26.29.86 → 8.8.8.8 | ICMP
Echo Request: 172.26.29.86 → 8.8.8.8

===== RESUMO ESTATÍSTICO DA CAPTURA =====
Interface: wlo1
Total de pacotes validados e processados: 15
Tempo de captura: 0m 15s

dremar@83dremar:~
file:///83dremar/home/dremar
oss, time 33796ms
pipe 4
dremar@83dremar:~$ ping 8.8.8.8
PING 8.8.8.8 (8.8.8.8) 56(84) bytes of data.
64 bytes from 8.8.8.8: icmp_seq=1 ttl=113 time=35.7 ms
64 bytes from 8.8.8.8: icmp_seq=2 ttl=113 time=53.4 ms
64 bytes from 8.8.8.8: icmp_seq=3 ttl=113 time=53.3 ms
64 bytes from 8.8.8.8: icmp_seq=4 ttl=113 time=54.3 ms
64 bytes from 8.8.8.8: icmp_seq=5 ttl=113 time=165 ms
64 bytes from 8.8.8.8: icmp_seq=6 ttl=113 time=129 ms
```

Figura 20 - Captura de mensagens ICMP Echo Request e Echo Reply.

6.2.2 Captura de tráfego DNS

Para gerar tráfego DNS foram realizadas consultas de domínio através do comando:

nslookup google.com

O sniffer conseguiu identificar corretamente os pacotes DNS e extrair os domínios presentes nas queries observadas.

Durante a captura foi possível observar queries DNS e respectivas respostas, permitindo identificar os domínios consultados e os endereços IP devolvidos pelo servidor DNS.

```
===== CONFIGURAÇÃO FINAL =====
Interface: wlo1
Pacotes: 10
Protocolo: dns
Filtro BPF: Nenhum
IP: Nenhum
MAC: Nenhum
Log: Desativado
=====

===== Packet Sniffer RC.TP2 =====
Interface: wlo1
Filtro BPF: Nenhum
Filtro protocolo: dns
Filtro IP: Nenhum
Filtro MAC: Nenhum
Log: Desativado
A capturar... usa CTRL+C para parar.
=====

[ #1 ] [ 2026-05-06 17:30:57 ] Len:74  TTL:64  | Ethernet/IPv4/UDP/DNS  | MAC: a8:41:f4:e6:e3:0d → d4:7f:35:16:e2:9f | IP: 172.26.29.86 → 193.137.16.145 | DNS Query: dominowdirect.adsrvr.org: 172.26.29.86:55532 → 193.137.16.145:53
[ #2 ] [ 2026-05-06 17:30:57 ] Len:137  TTL:62  | Ethernet/IPv4/UDP/DNS  | MAC: d4:7f:35:16:e2:9f → a8:41:f4:e6:e3:0d | IP: 193.137.16.145 → 172.26.29.86 | DNS Response: respostas=b'eur-bld-openpath.adsrvr.org.', 52.223.6.21, 3
[ #3 ] [ 2026-05-06 17:30:58 ] Len:84  TTL:64  | Ethernet/IPv4/UDP/DNS  | MAC: a8:41:f4:e6:e3:0d → d4:7f:35:16:e2:9f | IP: 172.26.29.86 → 193.137.16.145 | DNS Query: dominiofastlane.rubiconproject.com: 172.26.29.86:58528 → 193.137.16.145:53
[ #4 ] [ 2026-05-06 17:30:58 ] Len:84  TTL:64  | Ethernet/IPv4/UDP/DNS  | MAC: a8:41:f4:e6:e3:0d → d4:7f:35:16:e2:9f | IP: 172.26.29.86 → 193.137.16.145 | DNS Query: dominiofastlane.rubiconproject.com: 172.26.29.86:58835 → 193.137.16.145:53
[ #5 ] [ 2026-05-06 17:30:58 ] Len:153  TTL:62  | Ethernet/IPv4/UDP/DNS  | MAC: d4:7f:35:16:e2:9f → a8:41:f4:e6:e3:0d | IP: 193.137.16.145 → 172.26.29.86 | DNS Response: respostas=b'tagged-by.rubiconproject.net.akadns.net.', 69
[ #6 ] [ 2026-05-06 17:30:58 ] Len:165  TTL:62  | Ethernet/IPv4/UDP/DNS  | MAC: d4:7f:35:16:e2:9f → a8:41:f4:e6:e3:0d | IP: 193.137.16.145 → 172.26.29.86 | DNS Response: respostas=b'tagged-by.rubiconproject.net.akadns.net.', 26
[ #7 ] [ 2026-05-06 17:30:59 ] Len:189  TTL:64  | Ethernet/IPv4/UDP/DNS  | MAC: a8:41:f4:e6:e3:0d → d4:7f:35:16:e2:9f | IP: 172.26.29.86 → 193.137.16.145 | DNS Query: dominowistio-k8s-vidazoo-p-us-nyc3-external.vidazoo.service
[ #8 ] [ 2026-05-06 17:30:59 ] Len:184  TTL:62  | Ethernet/IPv4/UDP/DNS  | MAC: d4:7f:35:16:e2:9f → a8:41:f4:e6:e3:0d | IP: 193.137.16.145 → 172.26.29.86 | DNS Response: 193.137.16.145:53 → 172.26.29.86:54793
[ #9 ] [ 2026-05-06 17:31:01 ] Len:86  TTL:64  | Ethernet/IPv4/UDP/DNS  | MAC: a8:41:f4:e6:e3:0d → d4:7f:35:16:e2:9f | IP: 172.26.29.86 → 193.137.16.145 | DNS Query: dominiopages02.googleyndication.com: 172.26.29.86:41442 → 193.137.16.145:53
[ #10 ] [ 2026-05-06 17:31:01 ] Len:114  TTL:62  | Ethernet/IPv4/UDP/DNS  | MAC: d4:7f:35:16:e2:9f → a8:41:f4:e6:e3:0d | IP: 193.137.16.145 → 172.26.29.86 | DNS Response: respostas=2a00:1450:400e:004::2002: 193.137.16.145:53 → 172.26.29.86:41442

===== RESUMO ESTATÍSTICO DA CAPTURA =====
Interface: wlo1
Total de pacotes validados e processados: 10
Tempo de captura: 0m 4s

Distribuição de Tráfego por Camadas OSI:

[Camada 2 (Ligação de Dados)]
> Ethernet: 10 (100.0%)

[Camada 3 (Rede)]
> IPv4 : 10 (100.0%)

[Camada 4 (Transporte)]
> UDP : 10 (100.0%)

dremar@83dremar:~
$ nslookup google.com
Server: 127.0.0.53
Address: 127.0.0.53#53

Non-authoritative answer:
Name: google.com
Address: 172.217.23.238
Name: google.com
Address: 2a00:1450:400e:80a::200e
```

Figura 21 - Captura de tráfego DNS durante consultas de domínio.

6.2.3 Captura de tráfego HTTPS

Durante a navegação web foram observadas ligações HTTPS associadas à porta TCP 443. Este tipo de tráfego corresponde a comunicações seguras utilizadas por websites e aplicações modernas.

Apesar do conteúdo se encontrar cifrado, o sniffer conseguiu identificar corretamente:

- endereços IP envolvidos;
- portas utilizadas;
- hierarquia de encapsulamento;
- flags TCP associadas à comunicação.

Durante a captura foi possível observar diferentes pacotes TCP relacionados com o estabelecimento e troca de dados da ligação HTTPS.

```

@renar@3drenar:~/Desktop/universidade/redes/REDES_SHIFFER$ sudo python3 sniffer.py -i wlo1 --prot https -c 10
===== Packet Sniffer RC-TP2 =====
Interface: wlo1
Filtro BPF: Nenhum
Filtro protocolo: https
Filtro IP: Nenhum
Filtro MAC: Nenhum
Log: Desativado
A capturar... usa CTRL+C para parar.
=====
#1 ] [2026-05-06 17:34:26] Len:52   TTL:64 | Ethernet/IPv4/TCP/HTTPS | MAC: a8:41:f4:e6:e3:6d → d4:7f:35:16:e2:9f | IP: 172.26.29.86 → 99.84.9.47 | HTTPS: 172.26.29.86:41868 → 99.84.9.47:443 | tráfego cifrado | TCP ACK - co
#2 ] [2026-05-06 17:34:26] Len:52   TTL:64 | Ethernet/IPv4/TCP/HTTPS | MAC: d4:7f:35:16:e2:9f → a8:41:f4:e6:e3:6d | IP: 99.84.9.47 → 172.26.29.86 | HTTPS: 99.84.9.47:443 → 172.26.29.86:41868 | tráfego cifrado | TCP ACK - co
#3 ] [2026-05-06 17:34:26] Len:60   TTL:64 | Ethernet/IPv4/TCP/HTTPS | MAC: a8:41:f4:e6:e3:6d → d4:7f:35:16:e2:9f | IP: 172.26.29.86 → 69.173.156.138 | HTTPS: 172.26.29.86:56358 → 69.173.156.138:443 | tráfego cifrado | TCP
#4 ] [2026-05-06 17:34:26] Len:60   TTL:64 | Ethernet/IPv4/TCP/HTTPS | MAC: a8:41:f4:e6:e3:6d → d4:7f:35:16:e2:9f | IP: 172.26.29.86 → 69.173.156.138 | HTTPS: 172.26.29.86:56372 → 69.173.156.138:443 | tráfego cifrado | TCP
#5 ] [2026-05-06 17:34:26] Len:60   TTL:64 | Ethernet/IPv4/TCP/HTTPS | MAC: a8:41:f4:e6:e3:6d → d4:7f:35:16:e2:9f | IP: 172.26.29.86 → 69.173.156.138 | HTTPS: 172.26.29.86:56374 → 69.173.156.138:443 | tráfego cifrado | TCP
#6 ] [2026-05-06 17:34:26] Len:40   TTL:180 | Ethernet/IPv4/TCP/HTTPS | MAC: d4:7f:35:16:e2:9f → a8:41:f4:e6:e3:6d | IP: 69.173.156.138 → 172.26.29.86 | HTTPS: 69.173.156.138:443 → 172.26.29.86:56372 | tráfego cifrado | TCP
#7 ] [2026-05-06 17:34:26] Len:40   TTL:64 | Ethernet/IPv4/TCP/HTTPS | MAC: a8:41:f4:e6:e3:6d → d4:7f:35:16:e2:9f | IP: 172.26.29.86 → 69.173.156.138 | HTTPS: 172.26.29.86:56372 → 69.173.156.138:443 | tráfego cifrado | TCP
#8 ] [2026-05-06 17:34:26] Len:40   TTL:111 | Ethernet/IPv4/TCP/HTTPS | MAC: d4:7f:35:16:e2:9f → a8:41:f4:e6:e3:6d | IP: 69.173.156.138 → 172.26.29.86 | HTTPS: 69.173.156.138:443 → 172.26.29.86:56358 | tráfego cifrado | TCP
#9 ] [2026-05-06 17:34:26] Len:40   TTL:64 | Ethernet/IPv4/TCP/HTTPS | MAC: a8:41:f4:e6:e3:6d → d4:7f:35:16:e2:9f | IP: 172.26.29.86 → 69.173.156.138 | HTTPS: 172.26.29.86:56358 → 69.173.156.138:443 | tráfego cifrado | TCP
#10 ] [2026-05-06 17:34:26] Len:1290 TTL:64 | Ethernet/IPv4/TCP/HTTPS | MAC: a8:41:f4:e6:e3:6d → d4:7f:35:16:e2:9f | IP: 172.26.29.86 → 69.173.156.138 | HTTPS: 172.26.29.86:56372 → 69.173.156.138:443 | tráfego cifrado | TCP
===== RESUMO ESTATÍSTICO DA CAPTURA =====
Interface: wlo1
Total de pacotes validados e processados: 10
Tempo de captura: 0m 1s

Distribuição de Tráfego por Camadas OSI:

[Camada 2 (Ligação de Dados)]
> Ethernet: 10 (100.0%)

[Camada 3 (Rede)]
> IPv4 : 10 (100.0%)

[Camada 4 (Transporte)]
> TCP : 10 (100.0%)

[Camada 7 (Aplicação)]
> HTTPS : 10 (100.0%)

```

Figura 22 - Captura de tráfego HTTPS durante navegação web.

6.3. Demonstração dos Filtros

A aplicação implementa diferentes mecanismos de filtragem, permitindo reduzir o volume de tráfego apresentado e focar apenas os pacotes relevantes para análise.

Foram utilizados:

- filtros por protocolo;
- filtros por IP;
- filtros por MAC;
- filtros BPF.

Os filtros internos da aplicação são aplicados após a recepção do pacote, permitindo analisar o conteúdo e identificar protocolos específicos através da função `haslayer()`. Já os filtros BPF atuam diretamente no kernel do sistema operativo, fazendo com que apenas os pacotes correspondentes sejam entregues ao sniffer.

Para demonstrar o filtro por protocolo foi utilizado o comando:

```
sudo python3 sniffer.py -i wlo1 --proto ICMP -c 10
```

Durante este teste, apenas pacotes ICMP foram apresentados no terminal, ignorando o restante tráfego presente na interface.

```
[#1 ] [2026-05-06 20:14:21] Len:84   TTL:64 | Ethernet/IPv4/ICMP | MAC: a8:41:f4:e6:e3:6d → 14:ca:56:4d:ff:3b | IP: 192.168.1.218 → 8.8.8.8 | ICMP Echo
Request: 192.168.1.218 → 8.8.8.8
[#2 ] [2026-05-06 20:14:21] Len:84   TTL:116 | Ethernet/IPv4/ICMP | MAC: 14:ca:56:4d:ff:3b → a8:41:f4:e6:e3:6d | IP: 8.8.8.8 → 192.168.1.218 | ICMP Echo
Reply: 8.8.8.8 → 192.168.1.218
[#3 ] [2026-05-06 20:14:22] Len:84   TTL:64 | Ethernet/IPv4/ICMP | MAC: a8:41:f4:e6:e3:6d → 14:ca:56:4d:ff:3b | IP: 192.168.1.218 → 8.8.8.8 | ICMP Echo
Request: 192.168.1.218 → 8.8.8.8
[#4 ] [2026-05-06 20:14:22] Len:84   TTL:116 | Ethernet/IPv4/ICMP | MAC: 14:ca:56:4d:ff:3b → a8:41:f4:e6:e3:6d | IP: 8.8.8.8 → 192.168.1.218 | ICMP Echo
Reply: 8.8.8.8 → 192.168.1.218
[#5 ] [2026-05-06 20:14:23] Len:84   TTL:64 | Ethernet/IPv4/ICMP | MAC: a8:41:f4:e6:e3:6d → 14:ca:56:4d:ff:3b | IP: 192.168.1.218 → 8.8.8.8 | ICMP Echo
Request: 192.168.1.218 → 8.8.8.8
[#6 ] [2026-05-06 20:14:23] Len:84   TTL:116 | Ethernet/IPv4/ICMP | MAC: 14:ca:56:4d:ff:3b → a8:41:f4:e6:e3:6d | IP: 8.8.8.8 → 192.168.1.218 | ICMP Echo
Reply: 8.8.8.8 → 192.168.1.218
[#7 ] [2026-05-06 20:14:24] Len:84   TTL:64 | Ethernet/IPv4/ICMP | MAC: a8:41:f4:e6:e3:6d → 14:ca:56:4d:ff:3b | IP: 192.168.1.218 → 8.8.8.8 | ICMP Echo
Request: 192.168.1.218 → 8.8.8.8
[#8 ] [2026-05-06 20:14:24] Len:84   TTL:116 | Ethernet/IPv4/ICMP | MAC: 14:ca:56:4d:ff:3b → a8:41:f4:e6:e3:6d | IP: 8.8.8.8 → 192.168.1.218 | ICMP Echo
Reply: 8.8.8.8 → 192.168.1.218
[#9 ] [2026-05-06 20:14:25] Len:84   TTL:64 | Ethernet/IPv4/ICMP | MAC: a8:41:f4:e6:e3:6d → 14:ca:56:4d:ff:3b | IP: 192.168.1.218 → 8.8.8.8 | ICMP Echo
Request: 192.168.1.218 → 8.8.8.8
[#10 ] [2026-05-06 20:14:25] Len:84   TTL:116 | Ethernet/IPv4/ICMP | MAC: 14:ca:56:4d:ff:3b → a8:41:f4:e6:e3:6d | IP: 8.8.8.8 → 192.168.1.218 | ICMP Echo
Reply: 8.8.8.8 → 192.168.1.218

===== RESUMO ESTATÍSTICO DA CAPTURA =====
Interface: wlo1
Total de pacotes validados e processados: 10
Tempo de captura: 0m 27s

Distribuição de Tráfego por Camadas OSI:

[Camada 2 (Ligação de Dados)]
> Ethernet: 10 (100.0%)

[Camada 3 (Rede)]
> IPv4 : 10 (100.0%)
> ICMP : 10 (100.0%)
```

Figura 23 - Demonstração da utilização do filtro por protocolo ICMP.

Também foi realizado um teste utilizando um filtro BPF:

```
sudo python3 sniffer.py -i wlo1 -f "icmp" -c 10
```

Neste caso, o próprio sistema operativo filtrou os pacotes antes de serem processados pela aplicação, reduzindo a quantidade de tráfego entregue ao sniffer.

```
drenar@03drenar:~/Desktop/universidade/redes/REDES_SNIFFER$ sudo python3 sniffer.py -i wlo1 -f "icmp" -c 10
===== Packet Sniffer RC-TP2 =====
Interface: wlo1
Filtro BPF: icmp
Filtro protocolo: Nenhum
Filtro IP: Nenhum
Filtro MAC: Nenhum
Log: Desativado
A capturar... usa CTRL+C para parar.
=====

[#1 ] [2026-05-06 20:15:38] Len:84  TTL:64 | Ethernet/IPv4/ICMP | MAC: a8:41:f4:e6:e3:6d → 14:ca:56:4d:ff:3b | IP: 192.168.1.218 → 192.168.0.0 | ICMP
Echo Request: 192.168.1.218 → 192.168.0.0
[#2 ] [2026-05-06 20:15:39] Len:84  TTL:64 | Ethernet/IPv4/ICMP | MAC: a8:41:f4:e6:e3:6d → 14:ca:56:4d:ff:3b | IP: 192.168.1.218 → 192.168.0.0 | ICMP
Echo Request: 192.168.1.218 → 192.168.0.0
[#3 ] [2026-05-06 20:15:40] Len:84  TTL:64 | Ethernet/IPv4/ICMP | MAC: a8:41:f4:e6:e3:6d → 14:ca:56:4d:ff:3b | IP: 192.168.1.218 → 192.168.0.0 | ICMP
Echo Request: 192.168.1.218 → 192.168.0.0
[#4 ] [2026-05-06 20:15:41] Len:84  TTL:64 | Ethernet/IPv4/ICMP | MAC: a8:41:f4:e6:e3:6d → 14:ca:56:4d:ff:3b | IP: 192.168.1.218 → 192.168.0.0 | ICMP
Echo Request: 192.168.1.218 → 192.168.0.0
[#5 ] [2026-05-06 20:15:42] Len:84  TTL:64 | Ethernet/IPv4/ICMP | MAC: a8:41:f4:e6:e3:6d → 14:ca:56:4d:ff:3b | IP: 192.168.1.218 → 192.168.0.0 | ICMP
Echo Request: 192.168.1.218 → 192.168.0.0
[#6 ] [2026-05-06 20:15:43] Len:84  TTL:64 | Ethernet/IPv4/ICMP | MAC: a8:41:f4:e6:e3:6d → 14:ca:56:4d:ff:3b | IP: 192.168.1.218 → 192.168.0.0 | ICMP
Echo Request: 192.168.1.218 → 192.168.0.0
[#7 ] [2026-05-06 20:15:44] Len:84  TTL:64 | Ethernet/IPv4/ICMP | MAC: a8:41:f4:e6:e3:6d → 14:ca:56:4d:ff:3b | IP: 192.168.1.218 → 192.168.0.0 | ICMP
Echo Request: 192.168.1.218 → 192.168.0.0
[#8 ] [2026-05-06 20:15:45] Len:84  TTL:64 | Ethernet/IPv4/ICMP | MAC: a8:41:f4:e6:e3:6d → 14:ca:56:4d:ff:3b | IP: 192.168.1.218 → 192.168.0.0 | ICMP
Echo Request: 192.168.1.218 → 192.168.0.0
[#9 ] [2026-05-06 20:15:46] Len:84  TTL:64 | Ethernet/IPv4/ICMP | MAC: a8:41:f4:e6:e3:6d → 14:ca:56:4d:ff:3b | IP: 192.168.1.218 → 192.168.0.0 | ICMP
Echo Request: 192.168.1.218 → 192.168.0.0
[#10] [2026-05-06 20:15:48] Len:84  TTL:64 | Ethernet/IPv4/ICMP | MAC: a8:41:f4:e6:e3:6d → 14:ca:56:4d:ff:3b | IP: 192.168.1.218 → 192.168.0.0 | ICMP
Echo Request: 192.168.1.218 → 192.168.0.0

===== RESUMO ESTATÍSTICO DA CAPTURA =====
Interface: wlo1
Total de pacotes validados e processados: 10
Tempo de captura: 0m 25s

Distribuição de Tráfego por Camadas OSI:

[Camada 2 (Ligação de Dados)]
> Ethernet: 10 (100.0%)

[Camada 3 (Rede)]
> IPv4 : 10 (100.0%)
> ICMP : 10 (100.0%)

=====
```

Figura 24 - Demonstração da utilização de um filtro BPF com a expressão icmp.

6.4. Obstáculos Encontrados

Durante os testes realizados na interface real surgiram alguns obstáculos relacionados com a captura e análise do tráfego.

O principal problema encontrado esteve relacionado com o parâmetro count da função sniff() da biblioteca Scapy. Inicialmente, este parâmetro contabilizava todos os pacotes recebidos pela interface, incluindo aqueles que eram posteriormente rejeitados pelos filtros internos da aplicação. Como consequência, o sniffer podia terminar a captura antes de encontrar pacotes válidos do protocolo pretendido.

Para resolver este problema foi implementado um sistema próprio de validação e contagem de pacotes processados, permitindo considerar apenas os pacotes que passavam efetivamente nos filtros definidos.

Outro obstáculo encontrado foi a necessidade de utilizar permissões sudo para permitir o acesso à interface de rede em modo de captura.

Além disso, verificou-se que o tráfego numa interface real é significativamente mais complexo do que no ambiente controlado do CORE, devido à presença constante de tráfego multicast, broadcast e comunicações HTTPS cifradas. Apesar disso, o sniffer conseguiu identificar corretamente os principais protocolos observados durante os testes.

6.5. Resumo Estatístico

No final da captura, o sniffer apresenta automaticamente um resumo estatístico relativo aos protocolos identificados durante a monitorização da interface de rede.

Este resumo permite visualizar:

- número total de pacotes processados;
- tempo de captura;
- distribuição dos protocolos observados;
- organização dos protocolos pelas diferentes camadas do modelo OSI.

A informação estatística é gerada através da classe `Counter` da biblioteca `collections`, permitindo contabilizar automaticamente os protocolos identificados durante a execução da aplicação.

Na captura apresentada foi possível observar tráfego ICMP encapsulado sobre IPv4, distribuído pelas diferentes camadas do modelo OSI.

```
===== RESUMO ESTATÍSTICO DA CAPTURA =====
Interface: wlo1
Total de pacotes validados e processados: 10
Tempo de captura: 0m 13s

Distribuição de Tráfego por Camadas OSI:

[Camada 2 (Ligação de Dados)]
> Ethernet: 10 (100.0%)

[Camada 3 (Rede)]
> IPv4 : 10 (100.0%)
> ICMP : 10 (100.0%)

=====
```

Figura 25 - Resumo estatístico da captura efetuada na interface real.

Figura 26 - Exportação dos pacotes capturados para ficheiro CSV.

O formato JSON permite guardar a mesma informação de forma estruturada, facilitando o tratamento automático por outros programas ou scripts. Já o formato TXT preserva a saída textual apresentada no terminal, sendo útil para uma leitura rápida da captura.

Desta forma, a funcionalidade de logging permite manter um registo persistente do tráfego observado e facilita o tratamento posterior dos dados capturados.

8. Conclusão

O desenvolvimento deste trabalho prático permitiu concretizar com sucesso a implementação de um *Packet Sniffer* robusto e funcional, cumprindo integralmente os requisitos propostos no enunciado. A utilização da linguagem Python, aliada à biblioteca *Scapy*, revelou-se uma escolha adequada, permitindo abstrair a complexidade de manipulação de pacotes em bruto e focar o esforço na dissecação lógica dos cabeçalhos e na análise estrutural do tráfego.

Um dos pontos mais fortes da solução desenvolvida foi a implementação de uma arquitetura de filtragem híbrida. A combinação de filtros BPF (a atuar diretamente ao nível do *Kernel* para máxima otimização de performance e mitigação de "ruído" de rede) com filtros de Camada de Aplicação (processados em Python para uma dissecação profunda de protocolos como HTTP, DHCP ou DNS) dotou a ferramenta de enorme flexibilidade. Adicionalmente, a transição fluida entre o ambiente emulado (CORE) e a interface física real (Parte A e Parte B do projeto) comprovou a versatilidade do código concebido.

Em suma, a capacidade de transformar os conceitos teóricos do modelo OSI numa ferramenta analítica real e totalmente operacional permitiu consolidar os nossos conhecimentos em Redes de Computadores, dando por cumpridos, com rigor e sucesso, todos os objetivos estipulados para este trabalho prático.